1. Problem Scenario

MongoDB application stores customer information nd address information as separate data. This requires ultiple database operations to retrieve a customer's mplete information.

Current Design: Separate Collections

customers collection

```
"_id": 101,
"name": "Ravi Kumar",
"email": "ravi@example.com",
"phone": "9876543210"
```

addresses collection

```
{
    "customer_id": 101,
    "street": "Main Road",
    "city": "Guntur",
    "state": "Andhra Pradesh",
    "pin_code": "522001"
}
```

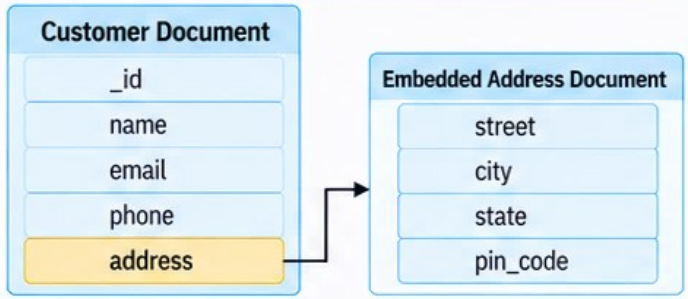
ssues with this design

- Requires multiple database queries.
- More application logic to combine data.
- Not efficient when customer and address are frequently accessed together.

2. Concept of Embedded Documents

- An **embedded document** is a document placed inside another document.
- It is useful when two pieces of information are closely related and are usually accessed together.
- In this case, the address belongs to one customer and is commonly needed with customer information.
- By embedding the address inside the customer document, we can retrieve both the customer and address details in a single database operation.

Structure of an Embedded Document



3. Redesigned Customer Document

The customer document now contains the address information as an embedded document.

New Design: Embedded Address

```
{
    "_id": 101,
    "name": "Ravi Kumar",
    "email": "ravi@example.com",
    "phone": "9876543210",
    "address": {
        "street": "Main Road",
        "city": "Guntur",
        "state": "Andhra Pradesh",
        "pin_code": "522001"
    }
}
```

Key Points

- Address is embedded inside the customer document.
- Customer and address can be retrieved together.
- Suitable when the address belongs to one customer.
- Reduces the need for separate lookups.

4. Before and After Comparison

Before: Separate Collections



customers collection

```
"_id": 101,
"name": "Ravi Kumar",
"email": "ravi@example.com",
"phone": "9876543210"
```



addresses collection

```
{
    "customer_id": 101,
    "street": "Main Road",
    "city": "Guntur",
    "state": "Andhra Pradesh",
    "pin_code": "522001"
}
```

rawbacks

- Multiple queries are required.
- Data is split across collections.
- Application has to combine the results.

After: Embedded Document



customers collection

```
{
    "_id": 101,
    "name": "Ravi Kumar",
    "email": "ravi@example.com",
    "phones": "9876543210",
    "address": {
        "street": "Main Road",
        "city": "Guntur",
        "state": "Andhra Pradesh",
        "pin_code": "522001"
    }
}
```

Advantages

- Single query retrieves complete customer information.
- Related data is stored together.
- Simpler data model and application logic.
- Efficient for commonly accessed information.

5. Example with Multiple Customers and Key Takeaways

Example: Multiple Customer Documents

Customer 1

```
{
    "_id": 101,
    "name": "Ravi Kumar",
    "email": "ravi@example.com",
    "address": {
        "street": "Main Road",
        "city": "Guntur",
        "state": "Andhra Pradesh",
        "pin_code": "522001"
    }
}
```

Customer 2

```
{
    "_id": 102,
    "name": "Sita Reddy",
    "email": "sita@example.com",
    "address": {
        "street": "MG Road",
        "city": "Vijayawada",
        "state": "Andhra Pradesh",
        "pin_code": "520010"
    }
}
```

When to Use Embedded Documents

- Address belongs to one customer.
- Address is usually accessed with customer information.
- Address is relatively small.
- Address does not need to be shared by many entities.

When Not to Use

- Address is shared by many customers
- Address data is very large.
- Address must be accessed independen
- Frequent updates to the same address across multiple customers.

Conclusion

By embedding the address inside the customer document, the application can retrieve customer and address information together in a single database operation. This design is suitable when the address is closely related to the customer and is frequently accessed together.

1. Scenario

A company stores customer information in MongoDB, but different customers have different sets of attributes.

2. Using Flexible Schema

- MongoDB allows each document to have its own structure.
- Different customers can have different fields based on the available information.
- Fields can be added, omitted, or nested as required, without changing a predefined schem

3. Customer 1 Document

This customer has basic information and an address.

```
{
    "_id": 101,
    "name": "Ravi Kumar",
    "email": "ravi@example.com",
    "phone": "9876543210",
    "address": {
       "street": "Main Road",
       "city": "Guntur",
       "state": "Andhra Pradesh",
       "pin_code": "522001"
    },
    "customer_type": "Regular",
    "created_at": "2025-09-05"
}
```

4. Customer 2 Document

This customer has additional fields such as date of birth, preferences, and loyalty details, but no addre

```
{
    "_id": 102,
    "name": "Sita Reddy",
    "email": "sita@example.com",
    "phone": "9123456789",
    "date_of_birth": "1998-04-12",
    "preferred_language": "Telugu",
    "newsletter_subscribed": true,
    "loyalty_points": 250,
    "interests": ["Electronics", "Books", "Travel"],
    "membership": {
       "level": "Gold",
       "joined_on": "2024-01-15"
    },
    "notes": "Prefers online shopping"
}
```

5. Explanation and Key Takeaways

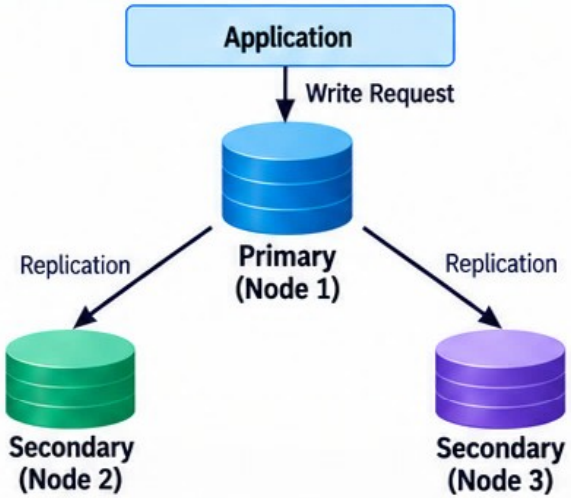
- Both documents are stored in the same MongoDB collection (e.g., customers).
- The documents have different sets of attributes.
- Customer 1 includes an address, while Customer 2 includes date of birth, interests, and loyalty information.
- This is possible because MongoDB uses a flexible schema, allowing each document to have its own structure.
- The application can store and retrieve only the fields that are relevant for each customer.

Benefits

- Supports diverse data requirements.
- No need to modify a predefined schema.
- Easier to handle optional or changing fields.
- Suitable for real-world applications where data varies between users.

1. MongoDB Replica Set (Three Nodes)

A replica set is a group of MongoDB servers that maintain copies of the same data.
It has one Primary node and one or more Secondary nodes.
In this example, the replica set contains three nodes (1 Primary and 2 Secondaries).
Write operations are sent to the Primary, which then replicates the changes to the Secondary nodes.



2. Understanding w = "majority"

- **w = "majority"** means the write must be acknowledged by a majority of the voting members in the replica set.
- For N voting members, majority = floor(N / 2) + 1.
- In a three-node replica set, the majority is 2.

Majority = floor(3 / 2) + 1 = 1 + 1 = 2

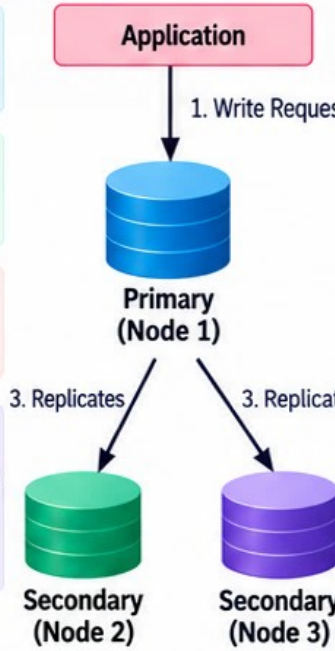
Therefore, at least 2 voting nodes must acknowledge the write before the operation is considered successful.

Majority Calculation for 3 Nodes

Step	Expression	Result
1	Total voting nodes (N)	3
2	N / 2	1.5
3	floor(N / 2)	1
4	floor(N / 2) + 1	2
Required acknowledgments (for w = "majority")		**2**

3. How a Majority Write Works

1. Application sends a write request (e.g., insert, update).
2. Primary receives the write operation.
3. Primary replicates the write to the secondary nodes.
4. The write is considered successful after acknowledgments are received from at least 2 voting nodes.



Success when at least 2 nodes acknowledge (w = "majority")

4. Different Scenarios in a Three-Node Replica Set

Scenario	Node 1 (Primary)	Node 2 (Secondary)	Node 3 (Secondary)	Total Acknowledgments	Write Result (w = "majority")	Explanation
nodes available ypical case)	✓	✓	✓	3	Success	3 ≥ 2
ne secondary available	✓	✓	✗	2	Success	Still a majority (2 ≥ 2)
imary unavailable ut new primary elected om remaining nodes)	✗	✓	✓	2	Success	After election, 2 nodes acknowledge (2 ≥ 2)
o nodes unavailable nly one node up)	✓	✗	✗	1	Failure	Not enough acknowledgments (1 < 2)

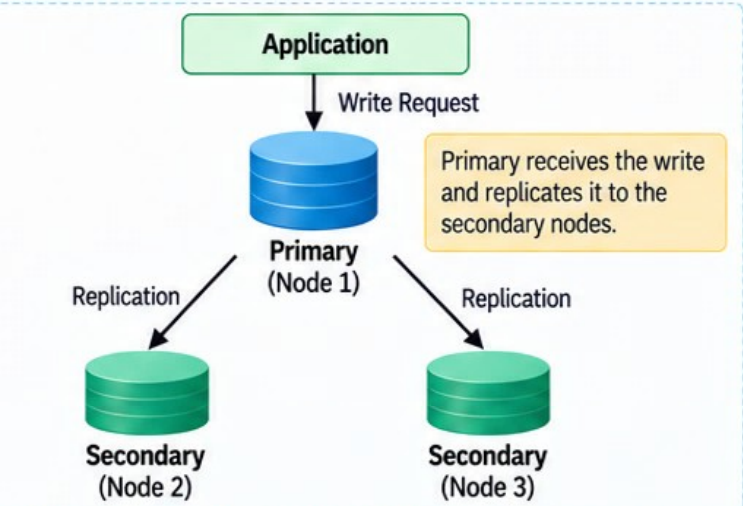
f a majority of 2 voting nodes cannot acknowledge the write, the operation will wait until the write concern timeout r eventually fail, depending on the configuration.

5. Key Takeaways

1. In a 3-node replica set, w = "majority" requires 2 acknowledgments.
2. The majority is calculated as floor(N / 2) + 1.
3. The write is successful when any two voting nodes acknowledge (e.g., primary + one secondary).
4. It provides stronger acknowledgment than w = 1 (which only requires primary acknowledgment).
5. It helps protect against data loss and ensures better durability.

1. MongoDB Replica Set with Three Nodes

A replica set is a group of MongoDB servers that maintain copies of the same data.
It has one Primary node and one or more Secondary nodes.
In this example, the replica set contains three nodes (1 Primary and 2 Secondaries).
Write operations are sent to the Primary, which replicates the changes to the Secondaries.



2. What Does w = "majority" Mean?

- The write concern **w = "majority"** means the write must be acknowledged by a majority of the voting members in the replica set.
- For N voting members, **majority** = floor(N / 2) + 1.
- In a three-node replica set, the majority is:

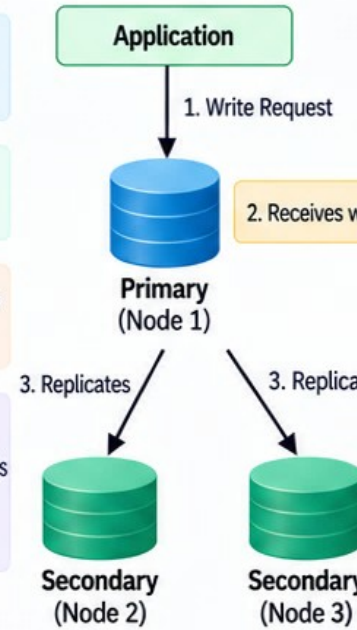
Majority = floor(3 / 2) + 1 = 1 + 1 = 2

Therefore, at least 2 voting nodes must acknowledge the write before the operation is considered successful.

Majority Calculation	
Total voting nodes (N)	3
N / 2	1.5
floor(N / 2)	1
floor(N / 2) + 1	2
Required acknowledgments (for w = "majority")	**2**

3. How a Write with w = "majority" Work

1. Application sends a write request (e.g., insert, update).
2. Primary receives the write.
3. Primary replicates the write to the secondary nodes.
4. The write is considered successful after acknowledgments are received from at least 2 voting nodes.



4. Success when at least 2 nodes acknowledge (w = "majority")

4. Different Scenarios

Scenario	Node 1 (Primary)	Node 2 (Secondary)	Node 3 (Secondary)	Total Acknowledgments	Write Result (w = "majority")	Explanation
nodes available pical case)	✓	✓	✓	3	Success	≥ 2 acknowledgments (3 ≥ 2)
e secondary available	✓	✓	✗	2	Success	Still a majority (2 ≥ 2)
mary unavailable t new primary elected m remaining nodes)	✗	✓	✓	2	Success	After election, 2 nodes acknowledge (2 ≥ 2)
o nodes unavailable ly one node up)	✓	✗	✗	1	Failure	Not enough acknowledgments (1 < 2)

a majority of 2 voting nodes cannot acknowledge the write, the operation will wait until the write concern timeout r eventually fail, depending on the configuration.

5. Key Takeaways

1. In a 3-node replica set, **w = "majority"** requires 2 acknowledgments.
2. The majority is calculated as **floor(N / 2) + 1.**
3. The write is successful when any **two** voting nodes acknowledge (e.g., primary + one secondary).
4. It provides stronger acknowledgment than **w = 1** (which only requires primary acknowledgment).
5. It helps protect against data loss and ensures better durability.
6. If two nodes are unavailable, the majority requirement cannot be satisfi and the write will not be acknowledged as successful.

Query Features in Document-Oriented Databases – Overview

hat are Query Features?

A query is a request to retrieve specific information from stored documents.
Document databases can query the contents of a document directly.
We do not always have to retrieve the entire document first.
Provides rich querying capabilities similar to relational databases, while retaining the flexibility of document storage.

ey Features

- Retrieve documents
- Filter documents using field values
- Select only required fields (projection)
- Count and sum data
- Query nested/embedded documents
- JSON-style query language
- Views and materialized views
- Query document contents without retrieving the whole document

Sample Document in a Collection

```
{
  "customerId": "C101",
  "name": "Ravi",
  "address": {
    "city": "Guntur",
    "state": "Andhra Pradesh"
  },
  "orders": [
    { "orderId": 501, "product": "Laptop", "amount": 60000 },
    { "orderId": 502, "product": "Mouse", "amount": 1000 }
  ]
}
```

What Can We Query?

Fields
- customerId
- name
- address.city
- orders.product
- orders.amount

Operations
- Find
- Filter
- Project (select fields)
- Sort
- Count
- Sum/Aggregation

Embedded Data
- Search inside nested objects
- Query arrays
- Use dot notation (e.g., address.city)

Document databases allow you to query the document, not just the key.

2. Basic Query Operations with Examples

Example 1: Retrieve All Documents

Find all orders in the collection.

SQL Equivalent
```
SELECT * FROM orders;
```

MongoDB Query
```
db.orders.find()
```

Sample Output
```
[
 { "orderId": 101, "customerId": "C101", "amount": 5000 },
 { "orderId": 102, "customerId": "C102", "amount": 8000 },
 { "orderId": 103, "customerId": "C101", "amount": 12000 }
]
```

Example 2: Filter Documents

Find all orders for customer C101.

SQL Equivalent
```
SELECT * FROM orders
WHERE customerId = 'C101';
```

MongoDB Query
```
db.orders.find({
  "customerId": "C101
})
```

Sample Data

orderId	customerId	amount
101	C101	5000
102	C102	8000
103	C101	12000
104	C103	7000

Query Result

orderId	customerId	amo
101	C101	500
103	C101	120

Example 3: Select Particular Fields (Projection)

Find C101's orders, but return only orderId and orderDate.

SQL Equivalent
```
SELECT orderId, orderDate
FROM orders
WHERE customerId = 'C101';
```

MongoDB Query
```
db.orders.find({
  { "customerId": "C101" },
  { "orderId": 1, "orderDate": 1 }
)
```

Query Result

orderId	orderDate
101	2026-09-01
103	2026-09-03

Example 4: Count and Sum

Count the number of orders and calculate the total amount.

MongoDB Queries
```
db.orders.count()
db.orders.aggregate([
  { $group: { _id: null, total: { $sum: "$amount" } } }
])
```

Result

Number of Orders	Total Amount
4	₹ 26,000

Querying Embedded Documents

er Document with Embedded Array

```
"orderId": 501,
"customerId": "C101",
"items": [
  {
    "product": {
      "name": "Laptop",
      "price": 60000
    },
    "quantity": 1
  },
  {
    "product": {
      "name": "Mouse",
      "price": 1000
    },
    "quantity": 2
  ]
```

Example: Find orders containing a product name "Refactoring"

SQL Equivalent (Multiple tables with JOIN)
```
SELECT o.orderId, p.name
FROM orders o
JOIN order_items i ON o.orderId = i.orderId
JOIN products p ON i.productId = p.productId
WHERE p.name LIKE '%Refactoring%';
```

MongoDB Query (Simpler)
```
db.orders.find({
  "items.product.name": /Refactoring/
})
```

Sample Data

orderId	customerId	items (product names)
501	C101	Laptop , Refactoring
502	C102	Python Programming
503	C101	Database Systems

Query Result

orderId	customerId
501	C101

Why it is simpler?
- Product information is embedded inside the order document.
- The database can directly query nested fields without using joins.

Document Structure for Querying

order
└ items[]
└ product
└ name

4. Views, Advanced Queries and Key Takeaways

CouchDB Views

- Views provide a way to query and analyze documents.
- There are two types of views:
  - Dynamic view – result is generated when needed.
  - Materialized view – result is precomputed and stored.
- Useful for complex queries and aggregation.

Example: Product Reviews View

Calculate number of reviews and average rating for each product.

Map function (concept)
```
emit(productId, { count: 1, rating: rating })
```

Reduce function (concept)
```
return {
  count: sum(count),
  average: sum(rating) / count
}
```

Use Case: Precomputed Results (Materialized View)

Reviews Collection
```
{ "product": "Laptop", "rating": 5 }
{ "product": "Laptop", "rating": 4 }
{ "product": "Laptop", "rating": 5 }
...
```

MapReduce / Computation →

Materialized View
```
{
  "product": "Laptop",
  "reviewCount": 10000
  "averageRating": 4.5
}
```

Instead of calculating every time, the precomputed result can be stored and quickly retrieved.

Query Features Summary

- Query document contents directly.
- Retrieve, filter, project, sort, count and sum.
- Query nested and embedded documents.
- JSON-style query language.
- Support for views and materialized views.
- Simpler queries when related data is embedded.
- More expressive than a simple key-value store.

Applications

- Banking (continuous access)
- E-commerce (high availability)
- Social M (user engage

Rich query capabilities make document databases suitable for real-world applications with complex and evolving data.

Suitable Use Cases of Document-Oriented Databases – Overview

y Document Databases Fit rtain Applications?

- Flexible data structures
- Handle non-uniform and evolving data
- Store related information in one document
- Support large volumes of data
- Easy horizontal scaling
- JSON-like documents
- Good for nested/embedded data

ey Idea

oose a document-oriented tabase when the application ta naturally fits as documents, anges frequently, and needs scale to large volumes.

ommon Suitable Use Cases

- Event logging
- Content management systems and blogging
- Web / real-time analytics
- E-commerce applications

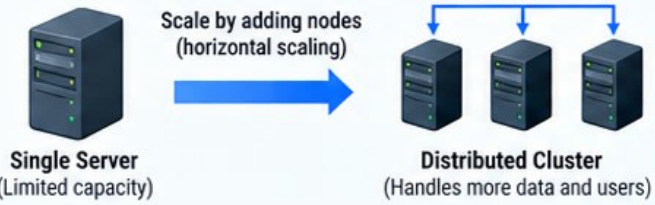
Document Model for Real-World Applications

```
{
  "id": "DOC1",
  "type": "example",
  "title": "Flexible Document",
  "attributes": {
    "field1": "value1",
    "field2": "value2"
  },
  "nestedData": [
    { "name": "Item A", "qty": 1 },
    { "name": "Item B", "qty": 2 }
  ]
}
```

Why It Works Well

- Different documents can have different fields.
- Schema can evolve over time.
- Nested information can be stored together.
- New attributes can be added easily.
- Can handle high data volume through replication and sharding.

From Single Server to Scalable Cluster



2. Use Case 1: Event Logging

What is Event Logging?

Recording events that happen in an application or organization, such as user logins, orders, payments, file uploads, and other activities.

Why Document Database is Suitable?

- Different event types have different structures.
- New event attributes can be added easily.
- Large volume of event data can be stored.
- Events can be distributed (sharded) by application or event type.

Examples of Events

- customer_logged
- order_processed
- payment_completed
- product_viewed
- user_registered
- file_uploaded

Different Event Documents (Flexible Structure)

Login Event

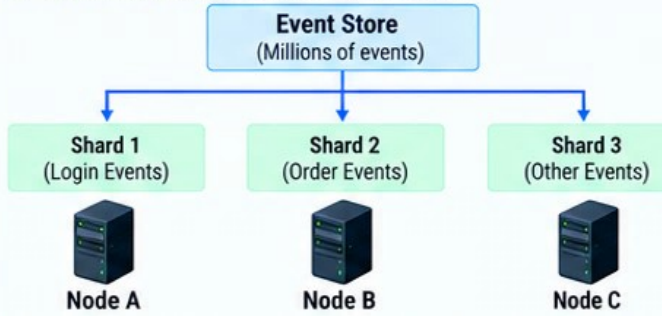
```
{
  "eventType": "customer_logged",
  "customerId": "C101",
  "loginTime": "10:30 AM",
  "ipAddress": "192.168.1.10",
  "device": "Mobile"
}
```

Order Event

```
{
  "eventType": "order_process
  "orderId": "O501",
  "customerId": "C101",
  "amount": 2500,
  "paymentMethod": "UPI",
  "status": "SUCCESS"
}
```

Scaling Event Data

Events can be sharded by application or event type.



Use Case 2: Content Management Systems and Blogging

at is a Content Management System (CMS)?

application used to create, store, manage, publish content such as articles, blog posts, vs, comments, web pages, images, and r profiles.

y Document Database is Suitable?

- Different content types can have different structures.
- Flexible structure supports evolving content.
- Comments and related information can be embedded.
- New fields (tags, images, videos, etc.) can be added easily.
- Works naturally with JSON-like documents.

pes of Content in a CMS

- Articles
- Blog posts
- News
- User profiles
- Comments
- Images and videos
- Announcements
- Product descriptions

Examples of Content Documents

Blog Post Document

```
{
  "title": "Introduction to NoSQL",
  "author": "Ravi",
  "content": "...",
  "tags": ["NoSQL", "Database"],
  "comments": 20
}
```

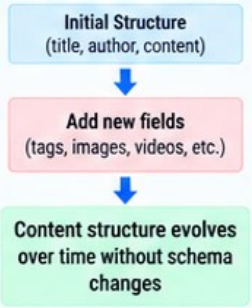
Blog Post with Extra Fields

```
{
  "title": "MongoDB Tutorial",
  "author": "Priya",
  "content": "...",
  "tags": ["MongoDB"],
  "video": "mongodb.mp4",
  "references": ["Database","JSON"]
}
```

Embedded Comments Example

```
{
  "title": "Introduction to Databases",
  "author": "Ravi",
  "content": "...",
  "comments": [
    {
      "user": "Anil",
      "comment": "Very useful",
      "date": "2026-09-01"
    },
    {
      "user": "Sita",
      "comment": "Good explanation",
      "date": "2026-09-02"
    }
]
}
```

Evolving Content Structure



4. Use Case 3: Web and Real-Time Analytics (and Others)

What is Web/Real-Time Analytics?

Collecting and analyzing information about user interactions with a website or application such as page views, unique visitors, clicks, searches, device type, browser, location, etc.

Why Document Database is Suitable?

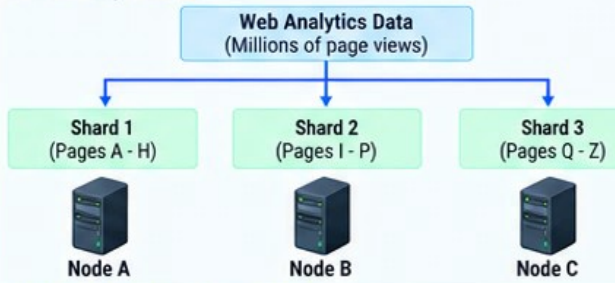
- Handles large volume of analytical data.
- New metrics can be added easily.
- Flexible structure for changing data.
- Supports real-time updates.
- Data can be distributed across multiple nodes using sharding.

Example Analytics Document (Page View)

```
{
  "userId": "U101",
  "page": "/products",
  "timestamp": "10:30:25",
  "device": "Mobile,
  "browser": "Chrome",
  "location": "India",
  "campaign": "SummerSale"
}
```

Scaling Web Analytics Data

Large volumes of page views and events can be distributed across multiple shards.



Other Suitable Use Case: E-Commerce

- Products can have different attributes (e.g., laptop vs. shirt).
- Orders can contain embedded items, shipping details, and payment info.
- Data model can evolve as business requirements change.
- High read and write volumes can be handled using replication and shard

Key Takeaway

Document-oriented databases are best suited for applications with flexible, evolving and high-volume data, where related information can stored together in a document and horizontal scaling is important.

Transactions in Document-Oriented Databases – Overview

at is a Transaction?

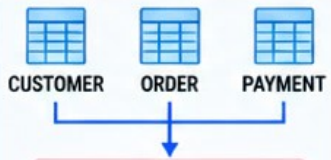
ransaction is a group of database erations that should be treated as one ical unit, so that the required changes ner all succeed or the system does not ve an incorrect partial result.

ey Points

- In traditional RDBMS: transactions can involve multiple tables and use COMMIT / ROLLBACK.
- In document databases: single-document operations are atomic (succeed or fail as a unit).
- Multi-document transactions were generally not available in early NoSQL systems, although some products (e.g., RavenDB) support them.
- Transaction capabilities depend on the particular database and its version.

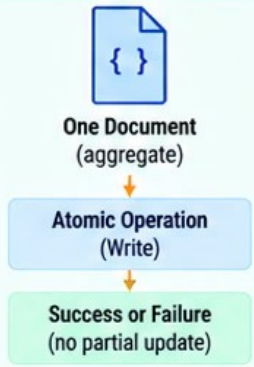
Traditional vs Document Database Transactions

Relational Database
(Multiple tables)



- Can involve multiple tables.
- Changes are committed or rolled back as a unit.
- Well-established transaction support (ACID).

Document Database
(Single document)



- Single-document operations are atomic.
- Related data can be kept together in one document.
- Multi-document transactions are more complex and may require application support.

2. How Transactions Work in Document Databases

Single-Document Atomic Update

A write to a single document is treated as one atomic operation.

```
{
  "orderId": 501,
  "customerId": 101,
  "status": "PLACED",
  "items": [
    { "product": "Laptop", "quantity": 1 }
  ]
}
```

Update status to SHIPPED
(single-document operation)

```
{
  "orderId": 501,
  "customerId": 101,
  "status": "SHIPPED",
  "items": [
    { "product": "Laptop", "quantity": 1 }
  ]
}
```

The update either succeeds completely or fails. There is no partial update.

Multi-Document Transaction (More Complex)

A business operation may require changes to multiple documer
Coordinating these changes is more difficult.



1. Create order document
2. Reduce inventory quantity
3. Record payment
4. Update customer document

- If one operation succeeds and another fails, the application must handle recovery (retry, compensating actions, etc.).
- Some document databases provide multi-document transactio but this was generally not available in early NoSQL systems.

Examples of Transactions in Document-Oriented Databases

mple 1: E-Commerce Order ngle-Document Transaction)

ating the order status is a single- ument operation and can be atomic.

```
orderId": 501,
customerId": 101,
status": "PLACED",
items": [
  { "product": "Laptop", "quantity":1 }
```

Update status to SHIPPED

```
orderId": 501,
customerId": 101,
status": "SHIPPED",
items": [
  { "product": "Laptop", "quantity": 1 }
```

ingle-document update is atomic and either succeeds or fails.

Example 2: Bank Account (Multiple-Document Transaction)

Transferring money between two accounts involves two different documents.

Account A

```
{
  "accountNo": "A101",
  "balance": 50000
}
```

- ₹10,000

Account B

```
{
  "accountNo": "B202",
  "balance": 20000
}
```

+ ₹10,000

One Business Transaction
(Transfer ₹10,000)

1. Update Account A (debit)
2. Update Account B (credit)

If Account A is updated but Account B is not, the system is left in an incorrect state.
This requires coordination and is more complex Than a single-document operation.

Example 3: Shopping Cart (Single-Document Transaction)

Updating the quantity of an item in a cart document is a single-document operation.

```
{
  "cartId": 1001,
  "customerId": 101,
  "items": [
    { "product": "Laptop", "quantity": 1 },
    { "product": "Mouse", "quantity": 2 }
  ]
}
```

Increase mouse quantity to 3

```
{
  "cartId": 1001,
  "customerId": 101,
  "items": [
    { "product": "Laptop", "quantity": 1 },
    { "product": "Mouse", "quantity": 3 }
  ]
}
```

This is an atomic single-document update.

4. Key Concepts, Trade-offs and Takeaways

Important Concepts

Concept	Explanation
Atomicity	A single-document write succeeds or fails as a unit.
Transaction	A logical unit of work consisting of one or more operations.
Single-document transaction	Supported and atomic in document databases.
Multi-document transaction	More difficult; generally not available in early NoSQL systems (some products support it).
WriteConcern	Specifies how many replica nodes must acknowledge a write. (Not the same as a transaction.)
Read consistency	Determines which replica is used for reads (primary or secondary).
Aggregate design	Keep related data together in one document to simplify atomic operations.

Consistency vs Performance Trade-off

Stronger Consistency

- More replicas must acknowledge the write
- More network coordination Higher latency
- Lower write performance

Trade-off

Higher Performanc

- Fewer replicas require Faster writes
- Replicas may tempora have different data
- Acceptable for some applications

Key Takeaways

- Single-document operations are atomic.
- Multi-document transactions are more complex and may require application logic or database-specific support.
- Design documents based on how the data is accessed (aggregate mo
- WriteConcern provides write safety, not a full transaction.
- Choose the level of consistency based on business requirements (e.g., strong for banking, moderate for e-commerce, relaxed for social me
- Document databases encourage developers to model related data tog to make atomic operations easier.

When Not to Use Document-Oriented Databases – Overview

ey Points

A document-oriented database is not suitable for every application.

Avoid it when requirements conflict with the strengths of the document model.

Two major situations:

- Complex transactions spanning different operations or documents.
- Queries against constantly varying aggregate structures.

In such cases, a relational database or another data model may be more appropriate.

Document Database vs. Situations Where It May Not Fit

Good Fit
(Use Document Database)



- Flexible structure
- JSON-like documents
- Embedded related data
- Evolving attributes
- Horizontal scaling
- High-volume data

Not a Good Fit
(Consider another database)



- Complex multi-document transactions
- Frequently changing aggregate structures
- Many unpredictable queries across different relationships
- Strong consistency requirements across multiple operations

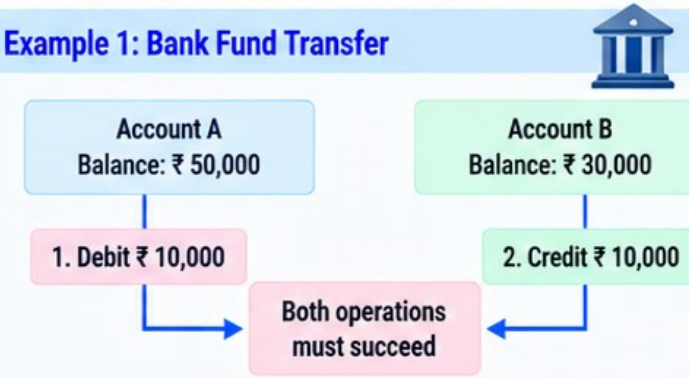
Choose the right tool based on application requirements, not just the technology.

2. Complex Transactions Spanning Different Operations

Why This Can Be a Problem

- A business operation may need to modify multiple documents.
- All changes must succeed or fail together (atomicity).
- This type of requirement is difficult in many document databases.
- Some document databases (e.g., RavenDB) do support these transactions, but it is not common across all systems.

Example 1: Bank Fund Transfer



If the second operation fails:

Account A	Account B
₹ 40,000 (debited)	₹ 30,000 (unchanged)

Data becomes inconsistent!

Complex Transactions in E-Commerce (Example 2)

ing an Order Involves Multiple Operations

1. Create order document
2. Reduce inventory
3. Record payment
4. Update customer order history

equirement

l operations must succeed or fail gether. If any step fails, the entire nsaction should be rolled back.

Multiple Documents Are Involved



Possible Failure Scenario

1. Create Order – Success
2. Reduce Inventory – Success
3. Record Payment – Failure

Should rollback all changes (order, inventory, etc.)

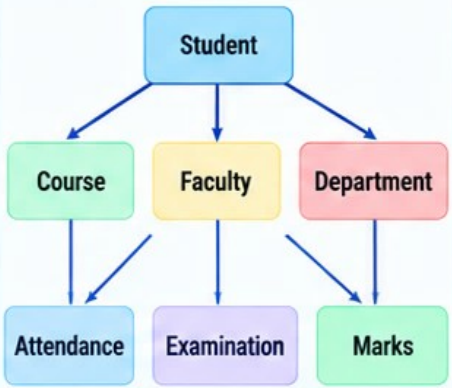
If such multi-document transactions are required frequently, a document database may not be the best choice.

4. Queries Against Varying Aggregate Structure (Example 3)

Why This Can Be a Problem

- When the aggregate structure changes frequently, it becomes difficult to design and maintain suitable documents.
- The application may need many different queries across relationships.
- Data may need to be stored at a lower level of granularity (essentially normalized).
- In such cases, a relational database is often more suitable.

Example 3: University Database



Different Query Requireme

- Find all students in CSE.
- Find all students taught by Professor X.
- Find students who scored m than 80 in DBMS.
- Find students who attende more than 90% of classes a scored above 80.
- Find students enrolled in Course A and Course B.
- Find students by departm faculty, subject, etc.

If the application requires constantly changing queries against different aggregates, a document database may not work well.

1. Document Database – Overview

Document Database is a type of NoSQL database that ores data as documents rather than rows and columns. ocuments are usually represented in formats such as JSON BSON and can contain simple fields, nested objects, and arrays.

ey Characteristics

- Stores data as documents (JSON/BSON/XML)
- Flexible schema (different documents can have different fields)
- Supports nested data and arrays
- Self-describing (field names and values are stored together)
- Reduces the need for joins
- Suitable for semi-structured and unstructured data

xamples of Document Databases



MongoDB



CouchDB



Couchbase

2. What is a Document?

A **document** is a collection of related data stored as a single unit. It can contain simple values, nested objects, and arrays.

Example: Customer Document (MongoDB)

```
{
    "_id": 101,
    "name": "Ravi",
    "email": "ravi@example.com",
    "phone": "9876543210",
    "address": {
        "city": "Guntur",
        "state": "Andhra Pradesh"
    },
    "skills": ["Python", "Java", "MongoDB"]
}
```

Points to Note

- The entire structure is one document.
- It contains simple fields (name, email), a nested object (address), and an array (skills).
- A document can represent a complex real-world object naturally.

3. What is a Row in an RDBMS?

An **RDBMS** stores data in tables. Each horizontal record in a table is called a row. The table has a predefined set of columns, and each r contains values for those columns.

Example: Customer Table (RDBMS)

CUSTOMER

Customer_ID	Name	Email	Phone
101	Ravi	ravi@example.com	9876543210
102	Sita	sita@example.com	9123456789

Related Information in Separate Tables

ADDRESS

Customer_ID	City	State
101	Guntur	Andhra Pradesh
102	Vijayawada	Andhra Pradesh

CUSTOMER_SKILLS

Customer_ID	Skill
101	Python
101	Java
102	Machine Learni

4. Document vs Row – Side by Side Comparison

Document (MongoDB)

```
"_id": 101,
"name": "Ravi",
"email": "ravi@example.com",
"phone": "9876543210",
"address": {
    "city": "Guntur",
    "state": "Andhra Pradesh"
},
"skills": ["Python", "Java", "MongoDB"]
```

One document contains all related information.
Supports nested objects and arrays.
Flexible schema (different documents can have different fields).
Self-describing structure.

Row (RDBMS)

CUSTOMER

Customer_ID	Name	Email	Phone
101	Ravi	ravi@example.com	9876543210

ADDRESS

Customer_ID	City	State
101	Guntur	Andhra Pradesh

CUSTOMER_SKILLS

Customer_ID	Skill
101	Python
101	Java
101	MongoDB

- Information is divided across multiple tables.
- Relationships are represented using keys.
- Requires joins to retrieve complete information.
- Columns are predefined.

5. Key Differences and Summary

Feature	Document (NoSQL)	Row (RDBMS)
Data model	Document	Table
Basic unit	Document	Row
Schema	Flexible	Predefined
Nested data	Naturally supported	Usually represented using related tables
Arrays	Naturally supported	Usually represented using additional rows
Relationships	Embedded or referenced	Primary and foreign keys
Joins	Often minimized	Commonly used
Data organization	Hierarchical	Tabular
Best suited for	Semi-structured / complex data	Structured, consistent data

Summary

A document database stores related information as a flexible, self-contained document. A document can contain nested objects and arrays, whereas an RDBMS row is one record in a table with predefined columns. For example, customer information including address and skills can be stored in a single document in MongoDB, but in an RDBM it is typically split across multiple tables.

1. Document Database – Overview

Document Database is a type of NoSQL database that tores data in the form of documents rather than traditional ows and columns.

Documents are usually stored in formats such as JSON, BSON, or XML.

Each document contains related information as a single unit.

It is designed to handle semi-structured and unstructured data.

It provides a flexible schema, which allows different documents to have different fields.

Examples: MongoDB, CouchDB, Couchbase.

Key Advantages

- Flexible schema
- Natural representation of real-world objects
- Handles nested and hierarchical data
- Reduces the need for joins
- Suitable for rapidly changing data

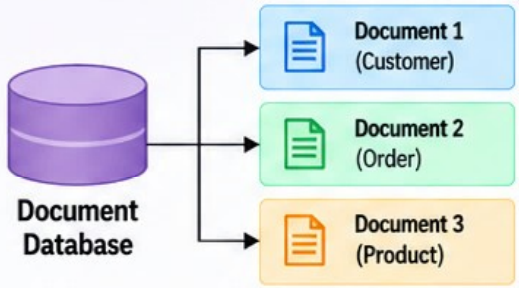
2. What is a Document?

A **document** is a collection of related data stored as a single unit. It contains field names and their values, and can include simple values, nested objects, and arrays.

Example: Customer Document (JSON)

```
{
    "customer_id": 101,
    "name": "Ravi",
    "email": "ravi@example.com",
    "phone": "9876543210",
    "address": {
        "city": "Guntur",
        "state": "Andhra Pradesh"
    }
}
```

Structure of a Document Database



3. Main Characteristics of a Document

1	**Self-Describing**	A document contains field names and their values, so the structure is clear from the document itself.	`{ "name": "Ravi", "age": 25 }`
2	**Flexible Schema**	Different documents in the same collection can have different fields.	**Doc 1:** `{ "id": 101, "name": "Rav` **Doc 2:** `{ "id": 102, "name": "Sit "phone": "9876543210`
3	**Supports Nested Data**	A document can contain nested objects.	`"address": { "city": "Guntu "state": "AP" }`
4	**Supports Arrays**	A document can contain arrays or lists.	`"skills": [ "Python", "Java "MongoDB" ]`
5	**Self-Contained**	A document can store most or all related information required for an application operation.	Order document with customer, items, shipping and payment details (see example in Panel 5)
6	**Aggregate-Oriented**	A document often represents a group of related information (aggregate) that is accessed or updated together.	Order → Customer, Items, Shipping, Payment
7	**Unique Identifier**	Each document has a unique identifier (e.g., _id in MongoDB).	`{ "_id": 101, "name": "Ravi`
8	**Queryable Fields**	Documents can be queried based on individual fields, including nested fields and array elements.	Find customers where address.city = "Guntur"

4. Document Database vs Relational Database

Feature	Document Database	Relational Database
Data model	Documents (JSON/BSON/XML)	Tables (rows and columns)
Basic unit	Document	Row
Schema	Flexible (schema-less or dynamic)	Predefined schema
Nested data	Naturally supported	Uses multiple tables
Arrays	Naturally supported	Usually represented with extra tables
Relationships	Often embedded or referenced	Primary and foreign keys
Joins	Often minimized	Commonly used
Schema changes	Easy to accommodate	May require schema modification
Best suited for	Semi-structured, hierarchical, rapidly changing data	Structured, consistent data

5. Example: Order Document in a Document Database

```
{
   "order_id": 1001,
   "customer": {
      "id": 101,
      "name": "Ravi"
   },
   "items": [
      { "product": "Laptop", "quantity": 1, "price": 50000 },
      { "product": "Mouse", "quantity": 2, "price": 3000 }
   ],
   "shipping": {
      "city": "Guntur", "state": "AP"
   },
   "payment": {
      "method": "Credit Card", "status": "Successful", "amount": 56000
   }
}
```

Key Takeaways

- A document database stores data as flexib self-contained documents.
- A document is self-describing, can contain nested objects and arrays, and is identifiec by a unique key.
- Documents are often used to represent aggregates (e.g., an order with customer, items, shipping and payment).
- This model reduces the need for joins and is suitable for applications with complex or frequently changing data.

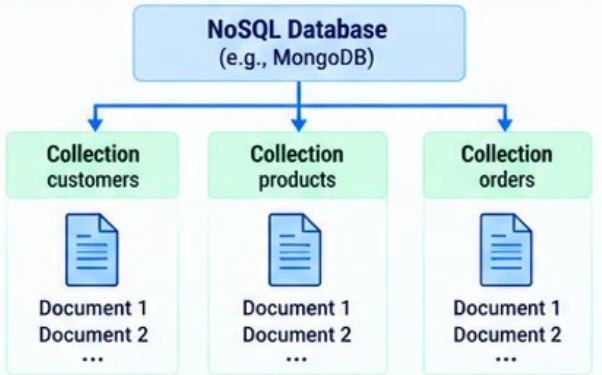
Document-Oriented Database – Overview and Key Features

at is a Document-Oriented Database?

ocument-oriented database is a NoSQL abase that stores data as documents pically in JSON, BSON or XML format) ner than rows and tables.

cuments are self-describing, can contain sted objects, arrays and simple values, d are stored in collections. They are signed to handle evolving data and le out over clusters of machines.

How Data is Organized



ajor Features

1. Self-describing documents
2. Flexible schema
3. Hierarchical data structure (nested objects/arrays)
4. Embedded documents
5. Rich query support
6. Collections and databases
7. Replication and high availability
8. Horizontal scalability (sharding)
9. Easy mapping to application objects
10. Ability to evolve the data model

Document Structure (JSON Example)

```
{
  "customerId": 101,
  "name": "Ravi",
  "city": "Guntur",
  "orders": [
    { "orderId": 501, "product": "Laptop", "amount": 60000 },
    { "orderId": 502, "product": "Mouse", "amount": 1000 }
  ]
}
```

Document databases provide a flexible and natural way to store and retrieve application data, especially when the data structure can change over time.

2. Flexible Schema and Hierarchical Data

Flexible Schema

Documents in the same collection can have different fields. New attributes can be added without modifying existing documents.

Customer Document 1

```
{
  "name": "Ravi",
  "age": 21,
  "city": "Guntur"
}
```

Customer Document 2

```
{
  "name": "Priya",
  "city": "Vijayawada",
  "skills": ["Python", "Java"]
}
```

Key Points

- No fixed schema for all documents in a collection.
- Different documents can contain different fields.
- Easy to add new attributes.
- Suitable for evolving and semi-structured data.

Hierarchical Structure with Embedded Documents

Documents can contain nested objects and arrays, allowi related data to be stored together.

Student Document Example

```
{
  "studentId": "23CS101",
  "name": "Ravi",
  "department": "CSE",
  "address": {
    "city": "Guntur",
    "state": "Andhra Pradesh"
  },
  "subjects": ["DBMS", "NoSQL", "Machine Learning"],
  "projects": [
    { "title": "Library Management", "technology": "Pytho
    { "title": "Student Portal", "technology": "MongoDB"
  ]
}
```

Benefits

- Natural representation of real-world objects.
- Related information can be retrieved together.
- Reduces the need for complex joins.
- Works well for data with nested relationships.

Document-Oriented Database Examples

mple 1: E-Commerce Application

e customer information with embedded es.

```
"customerId": 101,
"name": "Ravi",
"email": "ravi@example.com",
"address": {
  "city": "Guntur",
  "state": "Andhra Pradesh"
},
"orders": [
  { "orderId": 501,
    "product": "Laptop",
    "quantity": 1,
    "price": 60000
  },
  { "orderId": 502,
    "product": "Mouse",
    "quantity": 2,
    "price": 1000
]
```

hy it works well?

- Customer and orders can be stored together.
- Product attributes can vary.
- Easy to add new fields (e.g., preferences).
- Provides fast access to customer information.

Example 2: Student Profile System

Store student details with skills, projects and optional attributes.

```
{
  "studentId": "23CS101",
  "name": "Priya",
  "department": "IT",
  "address": {
    "city": "Vijayawada",
    "state": "Andhra Pradesh"
  },
  "skills": ["Python", "Java", "MongoDB"],
  "internships": [
    { "company": "ABC Tech", "duration": "3 months"
  ],
  "projects": [
    { "title": "Library Management", "tech": "Python"}
  ]
}
```

Why it works well?

- Flexible schema for different students.
- Easy to add new fields (e.g., certifications).
- Natural and readable structure.
- Suitable for managing evolving data.

Example 3: Event Logging / Web Analytics

Store different types of events with varying attributes.

Login Event

```
{
  "event": "login",
  "userId": 101,
  "time": "10:30"
}
```

Purchase Event

```
{
  "event": "purchase",
  "userId": 101,
  "productId": 501,
  "amount": 60000,
  "paymentMethod": "UPI"
}
```

Video Event

```
{
  "event": "video_play",
  "userId": 101,
  "videoId": 501,
  "duration": 240
}
```

Why it works well?

- Different event types can have different fields.
- New event types can be added easily.
- Suitable for real-time analytics and logging.
- Handles large volumes of semi-structured data.

4. Querying, Replication and Scalability

Rich Query Support

Fields inside documents (including nested fields) can be queried. Queries can filter, sort, count and search within documents.

Example Query (MongoDB-style)

```
// Find customers who purchased a Laptop
db.customers.find({ "orders.product": "Laptop" })

// Find a customer by ID
db.customers.find({ "customerId": 101 })

// Find products with price greater than 10000
db.products.find({ "price": { $gt: 10000 } })
```

Replication for High Availability

Multiple copies of the same data are maintained on different nodes (replica set).

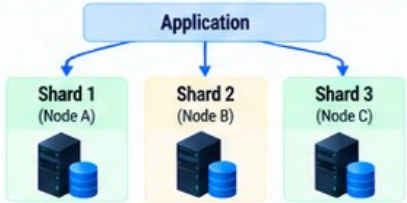


Benefits

- Data redundanc
- Failover suppor
- Read scaling
- Maintenance w reduced downti
- Disaster recove

Horizontal Scaling (Sharding)

Data can be distributed across multiple nodes by sharding, allowing the system to handle large datasets and high traffic.



Benefits

- Distributes data across nodes
- Supports large datasets
- Allows additional writable nodes
- Handles high read and write traffic
- Improves scalability and performance

Summary

- Query inside documents
- Flexible and evolving schema
- Hierarchical and embedded data
- Replication for availability
- Sharding for scalability
- Well-suited for modern applicatio such as e-commerce, content management and real-time analy